

Dockerfile Notes

Writing a Dockerfile, every key instruction, image layers, docker build and a complete worked example

1. What is a Dockerfile?

A **Dockerfile** is a plain text file containing the instructions (the same commands you would otherwise type by hand) needed to build an application image. The command **docker build** reads it and creates a Docker image automatically.

```
Dockerfile --(docker build)--> Image --(docker run)--> Container
```

First example, line by line

```
FROM ubuntu
LABEL maintainer="aditya"
RUN apt-get update
CMD ["echo", "Hello from aditya"]
```

Line	What it does	When it happens
FROM ubuntu	Start from the Ubuntu base image	Build time
LABEL maintainer=...	Adds author info as image metadata	Build time
RUN apt-get update	Runs the command and saves the result as a new layer	Build time
CMD [...]	Default command the container runs when it starts	Run time

```
docker build -t hello-aditya .      # build the image
docker run hello-aditya             # prints: Hello from aditya
```

Careful: In your notes the author line was written **MANINTAINER**. The correct keyword was **MAINTAINER**, but it is now **deprecated**; use **LABEL maintainer="name"**. Also, always type plain straight quotes (" ") in a Dockerfile; curly quotes copied from documents break CMD.

2. Image Layers and docker history

An image is built from **layers**. Instructions that change the file system (such as RUN and COPY) each add a layer; instructions like CMD and ENV only add metadata. Layers are read-only and cached, so unchanged steps are reused on the next build, which makes rebuilds fast.

Image = stack of read-only layers (top = last instruction)

CMD ["echo", "Hello from aditya"]	metadata only (0 B)
COPY test.txt .	adds /app/test.txt
WORKDIR /app	creates /app
RUN apt-get update	package lists
FROM ubuntu	base image layers

docker history - inspect the layers

Shows each layer of an image, the instruction that created it, and its size.

Syntax:

```
docker history IMAGE
```

Examples:

```
docker history hello-aditya
```

IMAGE	CREATED BY	SIZE
a1b2c3...	CMD ["echo" "Hello from aditya"]	0B
d4e5f6...	RUN apt-get update	~40MB
7a8b9c...	(base ubuntu layers)	~75MB

(sample output, exact values differ on your machine)

3. Dockerfile Instructions

Note: In a real Dockerfile a # comment must be on its **own line**. The comments written beside instructions in the small examples below are only explanations for you; do not copy them onto the same line.

FROM - choose the base image

Sets the base image. Every instruction after it builds on this image. A Dockerfile starts with FROM.

Syntax:

```
FROM <image>[:tag]
```

Examples:

```
FROM ubuntu                # latest ubuntu
FROM python:3.10-slim      # Python 3.10, small variant (pinned tag)
```

Tip: Pin a tag (python:3.10-slim) instead of relying on latest, so builds stay reproducible.

RUN - execute commands at BUILD time

Runs commands in a new layer on top of the current image and commits the result. Use it to install packages and prepare the image.

Syntax:

```
RUN <command>
```

Examples:

```
RUN apt-get update
RUN apt-get update && apt-get install -y curl  # chain with && = one layer
```

Tip: Chaining related commands with && keeps the layer count (and image size) down and avoids stale package lists.

CMD - default command at RUN time

Specifies what runs when a container starts. Rules: only **one** CMD per Dockerfile (if several exist, only the **last** one takes effect), and it can be **overridden** from the command line.

Syntax:

```
CMD ["executable", "param1", "param2"]  # exec form (preferred)
CMD command param1 param2                # shell form
```

Examples:

```
CMD ["echo", "Hello from aditya"]
```

```
docker run hello-aditya          # runs the CMD -> Hello from aditya
docker run hello-aditya echo "Bye"  # overrides CMD -> Bye
```

CMD vs RUN

	RUN	CMD
When it runs	While building the image	When a container starts
Purpose	Install/prepare things (apt-get, pip install)	Define the default process of the container
Result	Creates a new image layer	Stores metadata only, no layer content
How many	As many as needed	Only one counts (the last)
Overridable?	No (baked into the image)	Yes, via <code>docker run IMAGE other-command</code>

ENV - set environment variables

Sets environment variables available at build time and in running containers. They act as **default values** that can be overridden at runtime with `-e`.

Syntax:

```
ENV <key>=<value>
```

Examples:

```
ENV NAME=aditya
```

```
CMD ["sh", "-c", "echo Hello $NAME"]
```

```
docker run myimg # Hello aditya
```

```
docker run -e NAME=Rahul myimg # Hello Rahul (override at runtime)
```

Careful: Your notes used `ENV name = aditya` with spaces around the equals sign; write `ENV name=aditya` instead.

Careful: `CMD ["echo", "$name"]` will print the literal text `$name`. The exec form (JSON array) does not use a shell, so variables are not expanded. Use shell form `CMD echo $name` or `CMD ["sh", "-c", "echo $name"]`.

EXPOSE - document the port

Tells Docker (and readers) which port the container listens on. It is **documentation**; it does not publish the port by itself.

Syntax:

```
EXPOSE <port>
```

Examples:

```
EXPOSE 80 # app listens on port 80 (HTTP)
```

```
docker run -p 8080:80 myimg # host 8080 -> container 80 (this publishes it)
```

Note: To reach the app from your browser you must publish the port with `-p HOST:CONTAINER` at run time.

4. The docker build Command

Creates an image from a Dockerfile plus a **build context** (the folder whose files Docker is allowed to see during the build).

```
docker build [options] PATH | URL | -
```

Part	Meaning
PATH	Build context folder. <code>.</code> means the current directory.
-t name:tag	Name (and optional tag) for the resulting image
-f file	Use a Dockerfile with a different name or location. Default is a file called <code>Dockerfile</code> in the context root.

```
# 1. Default: Dockerfile in current folder, current folder is the context
docker build -t myapp .

# 2. Tag a version
docker build -t myapp:1.0 .

# 3. Custom Dockerfile name/location + a different context folder
docker build -f files/Dockerfile.dev -t image2 imagefiles/
# -f -> which Dockerfile to read (files/Dockerfile.dev)
# -t -> name of the image (image2)
# last argument -> build context (imagefiles/)
```

Careful: Your note `docker build -f files/text.txt -t image2/imagefiles` mixes things up: `-t` only takes the image *name*. The build context is the **last argument** (the PATH), as in the example above.

5. COPY and WORKDIR

COPY - copy files into the image

Copies files or folders from the **build context** (your machine) into the image file system. It can only copy files that are inside the build context.

```
Syntax:
COPY <src> <dest>

Examples:
COPY app/main.py /app/code/           # one file into a folder
COPY app/utils /app/code/utils        # a folder (its contents are copied)
COPY test.txt .                       # '.' = current WORKDIR
COPY ../secret.txt .                  # FAILS: outside the build context
```

WORKDIR - set the working directory

Creates the directory if it does not exist and makes it the current directory for all following instructions (RUN, CMD, COPY, ADD...). The path is **inside the container**, not on your host. It exists in the image at build time and in the container at run time.

```
Syntax:
WORKDIR /path

Examples:
WORKDIR /app           # create /app and move into it
COPY test.txt .        # lands at /app/test.txt
```

Worked example: what each step does

```
FROM ubuntu
RUN apt-get update
# creates /app and switches into it
WORKDIR /app
# local test.txt -> /app/test.txt
COPY test.txt .
# runs inside /app, prints the file
CMD ["cat", "test.txt"]
```

```
$ docker build -t wd-demo .
$ docker run -it wd-demo bash
root@a1b2c3d4e5f6:/app# pwd
/app
root@a1b2c3d4e5f6:/app# ls
test.txt
```

Multiple WORKDIR statements chain together when you use relative paths:

```
WORKDIR /a
WORKDIR b
WORKDIR c
# current directory is now /a/b/c
```

Tip: Best practice: always use **absolute paths** in WORKDIR (for example /app or /var/www/html) so the result is obvious.

6. Complete Example: Python App with COPY, WORKDIR, ENV and CMD

Project folder on your machine (the build context):

```
my-app/
|-- Dockerfile
|-- app/
|   |-- main.py
|   |-- utils/
|       |-- helper.py
```

app/utils/helper.py and app/main.py

```
# app/utils/helper.py
def greet(name):
    return f"Hello, {name}!"

# app/main.py
import os
from utils.helper import greet
print(greet(os.environ.get("NAME", "world")))
```

Dockerfile

```
# 1. base image with Python
FROM python:3.10-slim
# 2. default value for the app
ENV NAME=aditya
# 3. create and enter /usr/src/app
WORKDIR /usr/src/app
# 4. copy only the utils folder
COPY app/utils /usr/src/app/code/utils
# 5. copy only main.py
COPY app/main.py /usr/src/app/code/
# 6. move to where main.py lives
WORKDIR /usr/src/app/code
# 7. run the app when the container starts
CMD ["python", "main.py"]
```

Build and run

```
docker build -t myapp:1.0 .

docker run --rm myapp:1.0                # Hello, aditya!
docker run --rm -e NAME=Rahul myapp:1.0  # Hello, Rahul!   (ENV overridden)
docker run --rm myapp:1.0 python --version # CMD overridden  -> Python 3.10.x
docker run --rm -it myapp:1.0 bash        # shell opens in /usr/src/app/code
```

Note: Python can read NAME directly from the environment, so exec-form CMD works here. The shell-expansion problem only appears when the shell itself must expand \$NAME (as with echo).

7. Quick Reference and Common Mistakes

Instruction	Purpose	Runs at
FROM image:tag	Base image (first instruction)	Build
RUN cmd	Execute a command, save a layer	Build
COPY src dest	Copy files from build context into image	Build
WORKDIR /path	Set (and create) the working directory	Build + run
ENV key=value	Set default environment variable	Build + run
EXPOSE port	Document the listening port	Metadata
CMD ["exe", "arg"]	Default start command (one, overridable)	Run

- **Straight quotes only.** Curly quotes (copied from Word/PDF) make CMD and ENV fail.
- **JSON array for CMD uses double quotes:** ["python", "main.py"], never single quotes.
- **EXPOSE does not publish ports.** Add -p 8080:80 to docker run.
- **COPY sees only the build context.** Files outside the folder you pass to docker build cannot be copied.
- **Put rarely-changing steps first** (FROM, install dependencies) and your code last so Docker's layer cache is reused.
- **One CMD only.** A second CMD silently replaces the first.
- **Build with a tag** (-t name:tag) so you can find and run the image later.